

Kidnapping Problem: Autonomous SLAM Robot

Dhruv Madan Ryan Ranjitkar Kerry Xiao Luhai Tang

Department of Robotics Engineering, Worcester Polytechnic Institute, Worcester, MA, USA
{dmadan, rranjitkar, kxiao, ltang}@wpi.edu

Abstract—This paper presents the development of an implementation of SLAM (Simultaneous Localization and Mapping) for the TurtleBot3. The robot uses a LiDAR sensor to scan its environment and search for new frontiers in new areas of the map, and self-localizes to its environment based on a static map. A path planner based on the A* Algorithm and a custom heuristic calculates the path for the TurtleBot with a pure pursuit controller in place for the robot to navigate its environment. This project uses a ROS stack of both custom and preexisting nodes, including GMapping and AMCL (Adaptive Monte Carlo Localization) Algorithm.

Index Terms—SLAM, Mapping, Localization, Path finding, Pure pursuit, GMapping, AMCL, Particle filter, A*, ROS

I. INTRODUCTION

Many robotics applications now require the ability to understand and navigate through unknown environments without human intervention, whether for efficiency, accessibility, or safety. At the core of these abilities is SLAM, simultaneous localization and mapping. This part of robotics has slowly become a core aspect to much of robotic development, from self-driving cars, automation in industry, and small things like robotic vacuums. Path planning also plays a critical role in this, determining efficient, safe, and quick routes to target unexplored or designated locations.

This project marked the culmination of our work focusing on SLAM robotics. Working with a TurtleBot3, with previous work from each lab integrated and extended to autonomously explore and navigate an unknown maze. Using the gmapping package, real-time mapping and localization was implemented to construct a 2D occupancy grid of the environment. In this lab, practical implementation of frontier-based exploration, path planning, and obstacle avoidance and global navigation strategies were used.

II. METHODOLOGY

A. System Architecture

The system was built on the TurtleBot3 platform which helps integrate hardware and software for autonomous robotics. The TurtleBot3 platform combines ROS nodes for perception, mapping, planning, and motion control into an integrated system that enables the user to easily integrate their own packages and develop their own task and control structure.

Software

- **Operating System:** Ubuntu 20.04 and ROS Noetic

- **SLAM:** Implemented SLAM using the gmapping ROS package.
- **Localization:** Implemented using amcl (Adaptive Monte Carlo Localization) which utilizes a particle filter and LiDAR scan matching to estimate the robot's pose and position in a map.
- **Path Planning:** A custom A* algorithm computes paths on an inflated configuration space (C-space).
- **Frontier Detection:** A custom node identifies frontiers from the occupancy grid and scores them based on distance and region size.
- **Control:** A pure pursuit controller converts the planned path into smooth velocity commands for the robot's base.
- **Visualization:** RViz is used for real-time display of the robot's pose, sensor data, planned paths, and detected frontiers.

Node Interaction and Communication: The system is organized as a set of ROS nodes communicating via topics and services:

- The `FrontierFinder` node processes occupancy maps and publishes frontier goals.
- The `PathPlanner` node offers a service interface to generate paths between two poses.
- The `PathMovement` node consumes the path and executes motion commands to follow it using pure pursuit.
- Nodes for SLAM (gmapping), localization (amcl), and motion control run in parallel and exchange data via standardized ROS messages.

This modular architecture facilitates flexibility in testing and debugging, allowing individual components to be updated or replaced without disrupting the overall system. It also makes the design scalable to future enhancements.

B. ROS Navigation Stack

ROS provides a navigation stack that consists of packages for mapping, localization, and navigation—such as gmapping, amcl, and move_base.

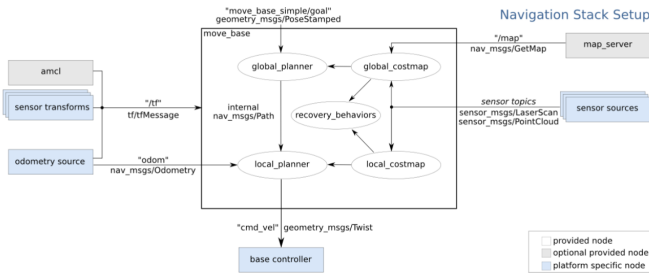


Fig. 1: The ROS Navigation Stack Interface

In this project, the `gmapping` and `amcl` packages were used for mapping and localization. For navigation and path planning, a custom A* algorithm was implemented instead of using the standard `move_base` package.

GMapping

The `gmapping` package in ROS performs SLAM (Simultaneous Localization and Mapping) using laser scan data and odometry to incrementally build a 2D occupancy grid map. It enables the robot to explore an unknown environment while constructing a map of its surroundings.



Fig. 2: Initial Startup of GMapping Node

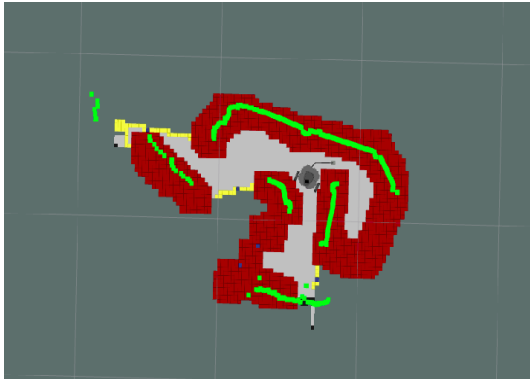


Fig. 3: RViz: C-Space (red), Path (line), Frontiers (Yellow), Centroids (Dark Blue)

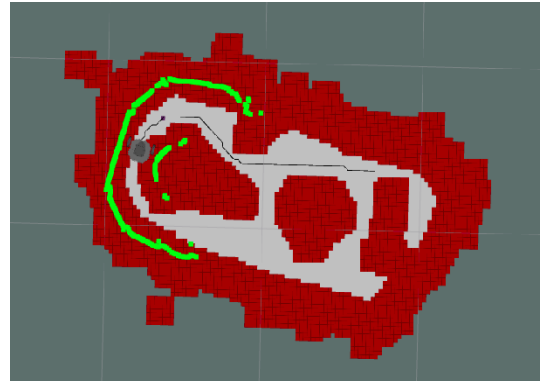


Fig. 4: Completion of Mapping, Path Back to Initial Starting Location

C. Frontier-Based Exploration

To enable autonomous mapping and navigation in an unknown environment, a frontier-based exploration strategy was implemented. This approach identifies regions at the boundary between known free space and unknown space (i.e., *frontiers*) and selects goal points for the robot to explore in a determined order.

1) *Frontier Detection*: The `FrontierFinder` node subscribes to the `/map` and `/odom` topics and maintains an updated occupancy grid and robot pose. Frontier detection follows these key steps:

- **Binary Map Construction**: The occupancy grid is converted into three binary maps:
 - *Unknown space*: Cells marked -1.
 - *Obstacles*: Cells marked 100, which are then dilated to prevent small gaps in the occupancy grid from being marked as frontiers.
 - *Free space*: Cells marked 0 in the occupancy grid, with dilated obstacles removed from the map.
- **Robot Location Handling**: A search is performed to ensure the robot's starting location is in valid free space:
 - If the robot is already in known free space, its current coordinate is used.
 - If the robot's location corresponds to an obstacle due to sensor error, the nearest valid free space cell is used instead.
- **Flood-Fill Reachability**: A flood-fill algorithm is executed from the robot's position to identify all reachable free-space cells. This prevents frontiers from being created in inaccessible or isolated regions.
 - This helps eliminate false frontiers caused by sensor noise or LIDAR blind spots.
- **Frontier Candidate Extraction**: The dilated unknown space is bitwise-ANDed with reachable free space to extract valid frontier candidates—free cells that border unknown space.
- **Clustering and Centroid Calculations**: Frontier candidates are clustered using `cv2.connectedComponentsWithStats`. The

centroid of each cluster is computed and relocated to the nearest valid frontier cell using a local expansion search.

- **Filtering and Scoring:** Each centroid is scored using a weighted heuristic:

$$\text{score} = \alpha \cdot \text{cluster size} + \beta \cdot \left(\frac{1}{\text{distance to robot} + 0.1} \right)$$

where α and β are tunable weights (default: $\alpha = 0$, $\beta = 10.0$). The highest-scoring centroid is selected as the robot's next goal.

2) *Goal Planning and Path Execution:* The `PathMovement` node orchestrates motion toward selected frontiers through the following process:

- It calls the `next_centroid` service provided by the `FrontierFinder` to obtain the next best exploration goal.
- It computes a collision-free path to this goal using the `plan_path` service, which invokes a custom A* algorithm running on an inflated configuration space (C-space).
- The robot follows the generated path using a pure pursuit controller, which adapts its linear and angular velocity based on path curvature and heading alignment.
- Upon reaching the goal, the cycle repeats until no frontiers remain.

This process continues until all valid frontiers are explored or the map is sufficiently complete.

D. Path Planning

To enable the robot to autonomously reach frontier centroids, we implemented a custom A* path planner node named `PathPlanner`. This node computes collision-free paths over an occupancy grid representing the robot's known environment.

1) *Configuration Space (C-Space) Construction:* The occupancy grid received from the `/map` topic is first processed to create a configuration space (C-space) that inflates obstacles. This inflation accounts for the physical footprint of the robot and introduces a buffer zone to reduce the risk of collision when navigating near walls. Obstacle dilation is performed using a kernel (in our case, a 7×7 , representing an inflation of 3 cells around the obstacle) through OpenCV. This ensured that narrow passages are ignored as they would be unsafe for the robot to path through.

2) *A* Implementation:* The A* algorithm is executed over the C-space. It operates on a 2D grid, where each cell corresponds to a pixel in the map, and its state is either free, occupied, or unknown. The algorithm uses a priority queue (min-heap) to expand nodes with the lowest estimated total cost:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost to reach node n from the start, and $h(n)$ is a heuristic estimate of the cost to reach the goal.

We employed a hybrid heuristic to improve navigation safety and efficiency:

- **Euclidean distance** to the goal
- **Distance-to-wall penalty** based on a precomputed distance transform, which finds the distance from the current cell to the nearest obstacle. This penalizes paths which hug the wall, allowing the robot to path through the middle of corridors and openings.
- **Angle penalty** to discourage frequent sharp turns

This allowed the planner to prefer smoother, safer routes that maintained a reasonable distance from obstacles while minimizing rotational effort.

3) *Path Representation and Publishing:* Once computed, the resulting path is formatted as a `nav_msgs/Path` message and published to the client node (`PathMovement`). Each waypoint in the path is expressed in the global map frame, allowing accurate visualization in RViz and enabling seamless integration with the pure pursuit controller.

4) *Service Architecture:* The planner operates as a ROS service named `plan_path`, conforming to the `nav_msgs/GetPlan` interface. Upon receiving a start and goal pose, the node converts these to grid coordinates, executes A*, and returns the planned path in world coordinates.

This modular structure decouples the planner from motion control and allows other nodes—such as the frontier explorer or AMCL tester—to request paths as needed.

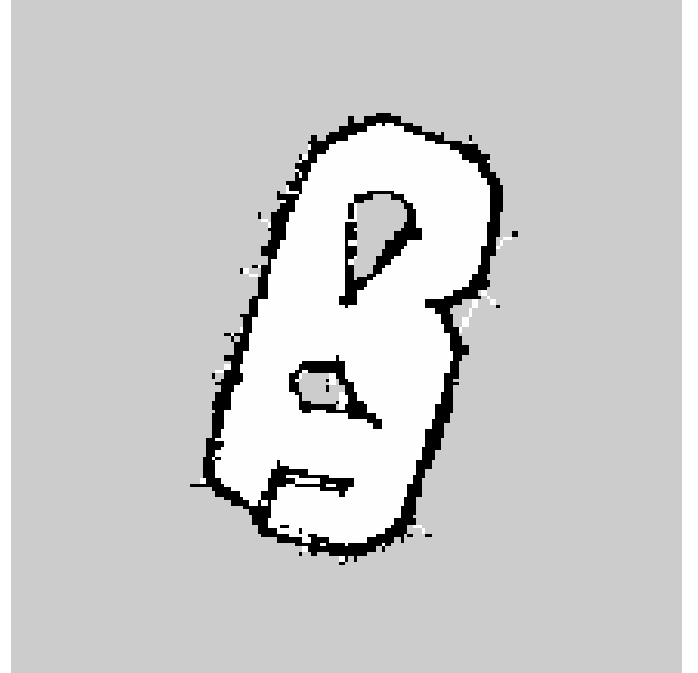


Fig. 5: Created Map through GMapping

Path Following & Pure Pursuit To track the calculated paths accurately and quickly, we implemented the Pure Pursuit algorithm in the `PathMovement` node. This geometric path tracking method constantly steers the robot towards a calculated lookahead point along the path. Our implementation operates as follows:

- **Lookahead Point Selection:** The robot continuously projects a circular region (radius: 0.1 m) centered on its current location. Then it searches for intersections between this circle and the segments of the global path. The first valid intersection that lies ahead of the robot is chosen as the lookahead point. If no such intersection is found, a fallback method projects a point at the desired distance along the path using along-track interpolation.
- **Control Command Calculation:** The angular velocity is calculated based on the heading error - the angle between the robot's current orientation and the direction to the look-ahead point. A proportional controller with dynamic saturation is used to prevent excessive turning speeds. The linear velocity is inversely scaled with the angular error to ensure stability during sharp turns and cornering.
- **Handling Sharp Turns:** If the required heading correction exceeds 135 degrees, the robot halts forward motion and initiates a rotation-in-place behavior to align itself with the new direction before resuming motion. This avoids erratic behavior when switching directions and enhances recovery in maze-like environments.
- **Pose Updating and Transformations:** The robot's current pose is transformed into the global map frame using TF2, ensuring all path-following computations remain consistent with the navigation coordinate system. A real-time visualization of the current pose and goal point is published using `GridCells` for debugging in RViz.
- **Completion Condition:** The robot continues following waypoints until it is within a small goal tolerance (0.05 m) of the final target, at which point it stops by publishing a zero-velocity command.

This custom implementation gave us full control over the robot's path-following behavior while maintaining modularity with our custom A* planner and frontier exploration modules. Compared to PID or move-base-based navigation, this approach offered more predictable behavior in narrow environments and allowed direct tuning of responsiveness and smoothness.

E. AMCL

The `amcl` package in ROS implements Adaptive Monte Carlo Localization, a probabilistic algorithm that uses a particle filter to estimate the robot's 2D pose on a known map. A custom node was developed to interface with `amcl`, initializing necessary publishers, subscribers, and services. The node subscribes to the robot's estimated pose and covariance published by `amcl`, as well as to a destination goal topic for receiving user-set goals from RViz. To evaluate localization confidence, the covariance matrix from the pose message is analyzed. Specifically, the trace of the top-left 2×2 submatrix (corresponding to x and y position uncertainty) is computed. A lower trace value indicates greater localization confidence. If the trace drops below a threshold of 0.025 for 50 consecutive readings, the robot is considered localized. At startup, the node triggers the `/global_localization` service, which distributes particles randomly across the map.

To localize without requiring physical movement, it uses the `/request_nomotion_update` service, forcing an update based solely on incoming sensor data—laser scan readings in the case of the TurtleBot3.

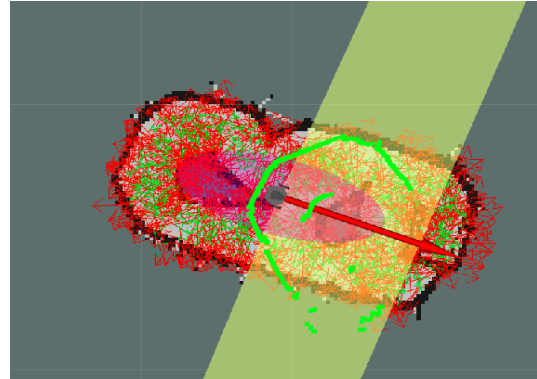


Fig. 6: Initial Startup of AMCL Node

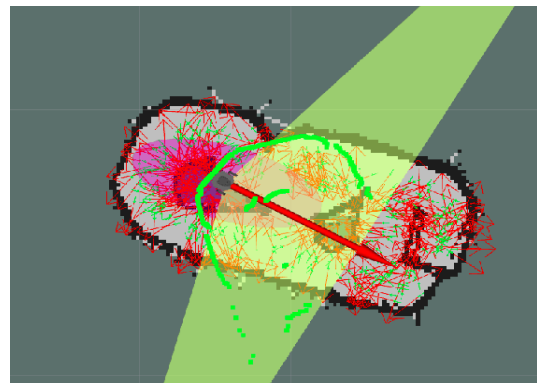


Fig. 7: Partway Through Localization

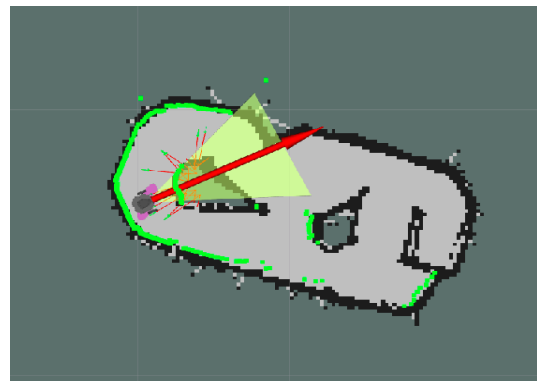


Fig. 8: Completion of Localization

Once localized, the robot stops by publishing a zero-velocity `Twist` message. The node then waits for a `PoseStamped` goal to be set in RViz. Upon receiving the goal, it calls the `plan_path` service, which uses the `nav_msgs/GetPlan` interface to compute an optimal path between the robot's current pose and the target destination. This approach was

chosen over continuous movement-based localization to minimize completion time. Since the timer only starts once the robot begins moving toward the goal, performing localization while stationary results in a faster completion time for the phase.

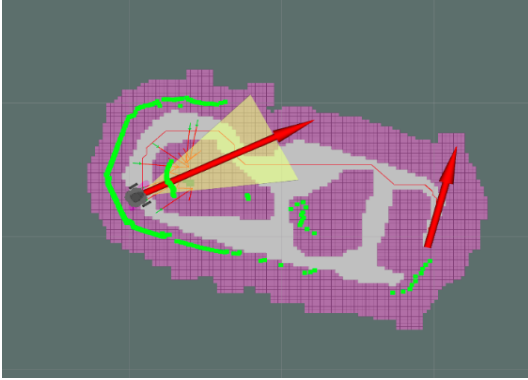


Fig. 9: Beginning of Pathing to Goal Location after Localization

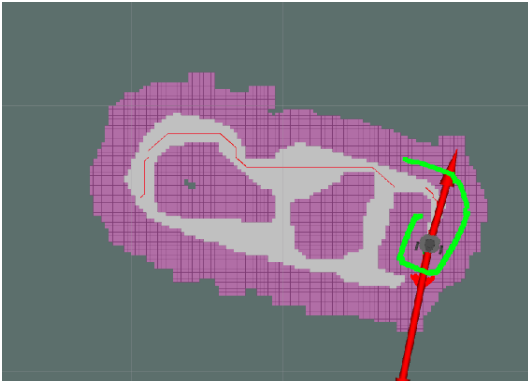


Fig. 10: Goal Location Reached after Localization

Simulation and Visualization Tools To support testing, debugging, and progress, we used RViz and Gazebo, two tools provided in the ROS ecosystem. *Rviz*: Rviz (ROS Visualization) was an essential tool throughout this project. It provided a real-time 3D interface to visualize sensor input and data, robot state, occupancy grids, and planned paths. During runs, RViz also displayed laser scan data, tracking the robot's estimated pose in the environment during AMCL and GMapping. Rviz also allowed us to manually set navigation goals through the "2D Nav Goal" interface, which made testing and evaluating the custom path planner significantly more efficient.

Gazebo: For simulating the robot in a virtual environment, Gazebo was used to validate our system before deploying it in the physical maze environment. It provided realistic sensor data, enabling us to test localization, navigation, and path planning algorithms in a risk-free setting. Using Gazebo saved significant time and effort, especially during the debugging of frontier exploration and path-following behavior, as it allowed repeatable scenarios and rapid iteration.

III. RESULTS

The system was successfully deployed on the Turtlebot3 in a controlled indoor maze environment. The robot was able to efficiently build a complete map of its environment while identifying and exploring all reachable frontiers and returning to its initial location with minimal intervention. During phase 3 of the demonstration, the robot encountered issues with correctly localizing itself in the map after being repositioned in the field. Due to these issues with localization, the robot was not able to navigate in the environment properly, thus needing significant human intervention.

Key Performance Outcomes and Observations

- **Exploration:** All reachable frontiers were visited and the robot returned back to its home position within 101 seconds with minimal human intervention.
- **Navigation Accuracy:** The robot followed generated paths with high precision, the pure pursuit controller enabled smooth turns and minimal pauses while path following.
- **Return to Start:** The robot was able to return to its start location within one radius of the robot's size.

IV. DISCUSSION

A. System Integration and Objectives

The primary objective of this project was to autonomously explore and map an unknown environment using a TurtleBot3, combining SLAM, frontier-based exploration, custom path planning, real-time path following, and localization based on the generated map. Our system successfully completed Phase 1 (localization) and Phase 2 (exploration) using ROS packages such as gmapping and amcl, integrated with our own A* planner and Pure Pursuit controller. While Phase 3 (returning to the start) was not completed due to final-stage localization issues, the robot demonstrated consistent autonomous behavior throughout mapping.

B. Frontier Detection

Frontier-based exploration was key to navigating unknown space efficiently. Our node detected frontiers by identifying reachable unknown-adjacent free cells using image-processing techniques in OpenCV. A scoring function, based on cluster size and distance from the robot, selected exploration goals. Initial difficulties with noisy frontiers and unreachable areas were resolved by refining reachability constraints and filtering small clusters.

C. Path Planning

We developed a custom A* planner to generate safe, efficient paths through the environment. By inflating obstacles in the configuration space and integrating distance-to-wall and angle-change penalties into the heuristic, the planner favored smoother paths through open areas. The modular service-based design allowed clean separation from other nodes such as localization and movement control.

D. Path Following

Path execution was handled by a Pure Pursuit controller that computed real-time angular and linear velocities based on a dynamic lookahead point. The robot maintained stability even in tight turns, and sharp directional changes were handled via in-place rotation logic. Overall, the controller provided smooth tracking performance across varying path geometries.

E. Localization and SLAM

We used `amcl` for localization and `gmapping` for SLAM. Localization was refined with motionless updates to improve consistency without requiring movement. RViz confirmed good alignment between estimated and actual pose during exploration. However, after full exploration, AMCL sometimes failed to maintain accuracy, contributing to the incomplete return in Phase 3.

F. Challenges and Limitations

Key challenges included TF synchronization errors, noisy initial frontiers, and tuning the SLAM parameters. These were gradually resolved, but system complexity made debugging time-intensive. A notable limitation was the lack of loop closure in exploration, which contributed to localization drift and ultimately prevented reliable execution of Phase 3.

G. Future Improvements

Future enhancements could include:

- Loop closure and map correction during long exploration sessions.
- Global frontier prioritization to improve coverage efficiency.
- Dynamic re-localization after exploration to support returning to known poses.
- Path smoothing or spline interpolation for less jagged trajectories.
- Dynamic Lookahead distance for a faster and more responsive traversal.

Overall, the project highlighted the importance of modular design, robust data synchronization, and real-time feedback visualization in developing autonomous robotic systems. The system's success in completing the full exploration and return sequence reinforces the value of combining custom logic with ROS's flexible framework.

V. CONCLUSION

This project showcases the integration of core SLAM principles and path planning strategies into a cohesive, autonomous mapping, path planning, and localization system. Using the Turtlebot3, and building on work done through previous labs, we successfully demonstrated the robot's ability to autonomously navigate an unfamiliar maze environment. Through the use of the `GMapping` package, we enabled real-time localization and map generation, creating a reliable 2D occupancy grid with RViz to guide subsequent exploration. The implementation of frontier-based exploration, global path

planning, and obstacle avoidance better expanded our understanding of these algorithms and parts of SLAM, but also highlighted the complexity of real-world robotics challenges. As SLAM and autonomous navigation continue to drive innovation in robotics—ranging from industrial automation to personal assistant devices—this project has provided us with a solid foundation in SLAM technologies and prepared us to contribute meaningfully to future advancements in the field.

REFERENCES

- [1] Purdue SIGBOTS, "Basic Pure Pursuit," *Purdue SIGBOTS Wiki*, [Online]. Available: <https://wiki.purduesigbots.com/software/control-algorithms/basic-pure-pursuit>
- [2] Open Source Robotics Foundation, "Services," *ROS Wiki*, [Online]. Available: <https://wiki.ros.org/Services>
- [3] Open Source Robotics Foundation, "AMCL: Adaptive Monte Carlo Localization," *ROS Wiki*, [Online]. Available: <https://wiki.ros.org/amcl>
- [4] OpenCV, "Morphological Transformations," *OpenCV Docs*, [Online]. Available: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html
- [5] Open Source Robotics Foundation, "Creating Msg and Srv," *ROS Wiki*, [Online]. Available: <https://wiki.ros.org/ROS/Tutorials/CreatingMsgAndSrv>
- [6] OpenCV, "Image Arithmetics," *OpenCV Docs*, [Online]. Available: https://docs.opencv.org/4.x/d0/d86/tutorial_py_image_arithmetics.html
- [7] S. Thrun, "Particle Filters in Robotics," in *Proceedings of the 17th Annual Conference on Uncertainty in AI (UAI)*, 2002. [Online]. Available: <https://robots.stanford.edu/papers/thrun.pf-in-robotics-uai02.pdf>

CONTRIBUTIONS

All members contributed equally to the design, implementation, and testing of the system.

Ryan Ranjitkar: Assisted in designing and implementing the FrontierFinder node, incorporating frontier detection and centroid extraction. Developed a Pure Pursuit controller for path tracking and helped optimize motion parameters for improved navigation performance. Managed ROS service integration within the system architecture, and overall communication methods.

Dhruv Madan: Worked on developing the frontier finder node and the Path planner node, including optimizing nodes for speed and efficiency in computing resources. Also worked on tuning heuristics for centroid picking and A* path planning.

Luhai Tang: Worked on developing the AMCL node, through a method tracking stable readings of a covariance trace under a given threshold. Created the launch files for all three phases, incorporating initializations of nodes, configurations and bring-ups. Worked on tuning movement profiles for the turtlebot for effective pure pursuit driving.

Kerry Xiao: Developed the AMCL Localization node and corresponding "Kidnapped Problem" nodes for Path Planner and Path Movement. Worked on tuning profiles for both the AMCL and GMapping nodes, as well as testing for speeds, heuristic, and frontier finding tuning.